CS2106 Introduction to Operating Systems
Lab 4 - Contiguous Memory Allocation
Week of 26th October 2026 (Initial Release)
Week of 2nd November, 2026 (Demos – Segment B)
Week of 9 November, 2026 (Demos – Segment A)

Instructions

a. For lab 4, the order of demonstration is B first: Students in segment B will demo in week 12, and students in segment A will demo in week 13.

Lab #	Release in week	Demo Week One | Demo Segment	Demo Week Two | Demo Segment
#04	11	12 | B	13 | A

b. The lab weightage is **7 marks.**

Introduction

In this lab we will look at implementing our own version of malloc and free, called "mymalloc" and "myfree", so that you can explore some of the issues in creating and implementing memory allocation algorithms.

There are two parts to this lab:

(1) Bitmap Allocation

In the first part you will implement a bitmap-based memory first-fit allocation algorithm. You will need to implement your own algorithms to scan a bitmap to allocate a suitable stretch of memory.

(2) Linked List Allocation

In the second part you will implement linked-list based worst-fit memory allocation algorithms. Instead of using a separate linked-list data structure to store information about the memory partitions, we will instead do so by having 'hidden' metadata stored in the memory alongside the partition.

1. Bitmap Memory Allocation

In this first section you will be allocating a first-fit memory allocation algorithm using bitmaps. Your bitmap will be stored in an array of unsigned char. Every bit in the bitmap represents information on the allocated/free state of one byte of memory.

Switch to "part1" directory. You will see a number of files:

Filename	Purpose
bitmap.h, bitmap.c	Header and source code files where you will implement the routines to manage the bitmap.
mymalloc.c, mymalloc.h	Header and source code files where you will implement your own versions of malloc and free, called mymalloc and myfree.
testmap.c	Test file to check that your bitmap routines work.
testmalloc.c	Demo file that uses your mymalloc and myfree.

We will now work on implementing a contiguous memory allocation scheme to service memory requests from mymalloc, using a bitmap to store information about the status of the memory. We will use **first-fit** allocation algorithm for this part.

1.1 Implement the Bitmap Algorithms

Start with bitmap.c and bitmap.h. You will find that some functions: print_map, allocate_map and free_map are already implemented for you. You will need to implement search_map to find the first-fit block of unallocated memory that fits the request, as well as set_map that modifies the bitmap (allocate_map and free_map call this function).

Function Name	Suggested implementation
search_map	Given a memory request, search through the bitmap to find the **first** free memory section large enough to fit the request. Returns the index of the start of the free memory section; -1 if no available hole is found.
set_map	Given input *index* and *length* and a *value* (which is 0/1) - Set bitmap[index] to bitmap[index + length – 1] to *value*.

Your job is to implement the two functions in the code so they can be used to perform first-fit contiguous memory allocation. Note that you will need to use some bit manipulation techniques to check each bit, as they are stored in a char array.

1.2 Testing your Bitmap Routines

You can test in isolation if your bitmap routines are written correctly in isolation before moving on to implementing our malloc routines by the following.

```
gcc testmap.c bitmap.c -o testmap
./testmap
```

If it is implemented correctly, you will see the following output; where all the expected and actual memory locations for each allocation match.

```
Length: 2, Expected: 2, Actual: 2
Length: 4, Expected: 5, Actual: 5
Length: 6, Expected: 5, Actual: 5
Length: 12, Expected: 26, Actual: 26
Length: 128, Expected: -1, Actual: -1

Allocating 2 bytes
BEFORE: 11001000 00011111 00001100 11000000 00000001
AFTER:  11111000 00011111 00001100 11000000 00000001

Allocating 12 bytes
BEFORE: 11111000 00011111 00001100 11000000 00000001
AFTER:  11111000 00011111 00001100 11111111 11111101

Length: 12, Expected: -1, Actual: -1
Freeing 12 bytes
BEFORE: 11111000 00011111 00001100 11111111 11111101
AFTER:  11111000 00011111 00001100 11000000 00000001

Length: 12, Expected: 26, Actual: 26
```

1.3 Implementing your Memory Manager

If your bitmap routines from 3.2 are working correctly, you can now implement your memory manager!

Open the mymalloc.c file and you will see some implemented functions for you. Your job is to implement the mymalloc and myfree functions.

Function Name	**Description**
mymalloc	Allocates memory and returns a pointer to the allocated memory, or NULL if no suitable memory is found.
myfree	Frees memory pointed to by ptr. Fails silently if ptr is NULL or does not point to a memory region created by mymalloc. (Note: This is different from free which crashes under such circumstances)

Note that mymalloc.h contains a defined constant for the size of the heap. We will use a small heap of 64 bytes (so it is easy to visualize).

You should allocate memory from the heap, simulated in mymalloc.c using an array of char, and track the allocation/free state of bytes by using the bitmap.

When implementing myfree, you will notice that the only provided parameter is the address of the allocated memory. However to free the memory in the bitmap, you will need to know exactly how much memory should be freed!
Looking at bitmap itself will not tell you this as two separate allocations in the bitmap that are located consecutively – e.g. 16 to 31 and 32 to 63 – will appear as a single contiguous allocated section of 16 to 63. Therefore, you will need to somehow keep track of this information. You may use the provided linked list library for this; alternatively, you can also code any other data structure to do this.

1.4 Testing Your Memory Allocation Routines

To test your memory allocation routines, compile and run using:

```
gcc testmalloc.c mymalloc.c bitmap.c -o testmalloc
./testmalloc
```

Note: If you use the linked list library, compile with:

```
gcc testmalloc.c mymalloc.c bitmap.c llist.c -o testmalloc
```

If your mymalloc and myfree are working properly, you should see:

```
Allocating 16 bytes to ptr1

Allocating 8 bytes to ptr2

Allocating 8 bytes to ptr3

Allocating 4 bytes to ptr4

Current state of bitmap:
11111111 11111111 11111111 11111111 11110000 00000000 00000000 00000000

Freeing ptr1
Freeing ptr3

Current state of bitmap:
00000000 00000000 11111111 00000000 11110000 00000000 00000000 00000000


Allocating 12 bytes to ptr5

Current state of bitmap:
11111111 11110000 11111111 00000000 11110000 00000000 00000000 00000000

Allocating 32 bytes to ptr1

Allocating 8 bytes to ptr1

Current state of bitmap:
11111111 11110000 11111111 11111111 11110000 00000000 00000000 00000000
Freeing all ptrs

Current state of bitmap:
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
```

You can also compare with the bitmap.out file provided in the bitmap directory.

DEMO 1: (3 marks)

Run ./testmalloc and demo to the TA that your code works.

2. Linked List Memory Allocation

Switch to the directory “part2”. Here, we will use a linked list to store information about the memory partitions allocated. However, instead of maintaining a separate linked list, we will instead store the data required for the linked list within the contiguous memory section (see below). Again, we will simulate the heap using a char array.

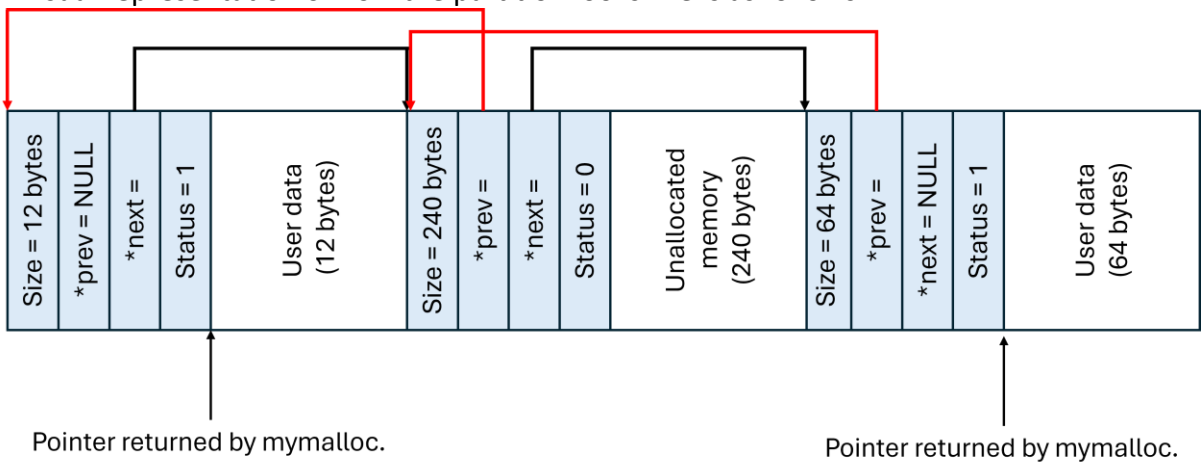
The Linked List Structure

The “heap” is structured as such. At the start of each partition of the heap, there is “hidden” metadata which stores the following

- Size of the partition
- Pointer to the start of the previous partition’s metadata
- Pointer to the next partition’s metadata
- Status of the partition (Occupied/free)

The memory immediately following the metadata is the memory allocated to the user request (or a free block of memory) equal to the size described in the metadata.

A visual representation of how the partition looks like is as follows.



The above shows three sample partitions: 2 occupied (i.e. allocated to mymalloc() requests) and 1 free (possibly allocated and then deallocated by user myfree() request). Essentially, you can imagine the linked list of the partition information is embedded in the partitions themselves. As with a normal doubly linked list, we keep track of pointers to the start of the next partition, and start of the previous partition. Note that the pointer that should be returned by mymalloc() points to the start of the user data portion, i.e. [Starting address of partition] + [offset] (which is the size of the meta data).

The partition meta data is defined as a partMetaInfo structure in the given mymalloc.h. We also keep a heapMetaInfo structure, which keeps track of some additional information, such as a pointer to the metadata of the first partition (essentially, like a pointer to the head of a linked list).

Implementing a Worst-Fit Allocation Algorithm

You will need to implement the code of mymalloc and myfree to correctly allocate memory using a **worst-fit** algorithm and to free allocated memory. Some code has been provided for

you, such as functions which will print out the partitions in the heap and some initialization code which can help you understand how the metadata are set up in the memory region.

DEMO 2. (4 marks)

Compile your memory manager with testmalloc.c and try running it.

```
gcc testmalloc.c mymalloc.c -o testmalloc
./testmalloc
```

You can compare it with test.out; in particular, check that the heap partitions of your implementation matches. (The start address will almost certainly be different.)

Demo to the TA that your code works by running `./testmalloc`